Task 14: Developing a More Complex Custom System Call in C

Objective:
The objective of this task is to develop a custom system call in the Linux kernel that performs a more complex operation than simple arithmetic. The system call accepts a string from user space, reverses the string safely in kernel space, and returns the reversed string back to user space. This task demonstrates kernel programming concepts, string manipulation in kernel space, and secure data transfer between user space and kernel space.

1. Introduction

Linux kernel and user space are strictly separated for security and stability. User programs cannot directly access kernel memory. System calls provide a controlled interface through which user programs request services from the kernel. In this task, a custom system call is implemented to reverse a string.

2. Overview of the Solution

The solution consists of two main components:
• Kernel-space system call implementation (written in C)
• User-space test program to invoke the system call

The kernel copies the string from user space, reverses it, and copies the result back to user space using safe kernel APIs.

3. Kernel-Space System Call Code

File Name: reverse_string_syscall.c

This file contains the implementation of the custom system call inside the Linux kernel.

```
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/uaccess.h>
#include <linux/string.h>

SYSCALL_DEFINE2(reverse_string,
                char __user *, input,
                char __user *, output)
{
```

```
    char kbuf[256];
    int len, i;

    if (copy_from_user(kbuf, input, sizeof(kbuf)))
    return -EFAULT;

    kbuf[255] = '\0';
    len = strlen(kbuf);

    for (i = 0; i < len / 2; i++) {
    char temp = kbuf[i];
    kbuf[i] = kbuf[len - i - 1];
    kbuf[len - i - 1] = temp;
    }

    if (copy_to_user(output, kbuf, len + 1))
    return -EFAULT;

    return 0;
}
```

4. Explanation of Kernel Code

• SYSCALL_DEFINE2 defines a system call with two arguments.
• copy_from_user() safely copies data from user space to kernel space.
• String reversal is performed inside kernel space.
• copy_to_user() safely copies the result back to user space.
• Direct access to user memory is avoided.

5. User-Space Test Program

File Name: reverse_string_user_test.c

This program runs in user space and invokes the custom system call.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <errno.h>
#include <string.h>

#define SYS_reverse_string 548

int main() {
```

```
    char *input = NULL;
    char *output = NULL;
    size_t buffer_size = 256;

    input = (char *)malloc(buffer_size);
    output = (char *)malloc(buffer_size);

    if (input == NULL || output == NULL) {
    perror("Memory allocation failed");
    return 1;
    }

    printf("Enter a string: ");
    fgets(input, buffer_size, stdin);

    if (syscall(SYS_reverse_string, input, output) < 0) {
    perror("System call failed");
    free(input);
    free(output);
    return 1;
    }

    printf("Reversed string from kernel: %s\n", output);

    free(input);
    free(output);

    return 0;
}
```

6. Data Transfer Between User Space and Kernel Space

copy_from_user() and copy_to_user() ensure secure data transfer between user space and kernel space, preventing invalid memory access and crashes.

7. Expected Output

Input: LinuxKernel
Output: lenreKxuniL

8. Conclusion

This task demonstrates the implementation of a custom Linux system call with safe memory handling and kernel-user interaction.